Lab 4 - Part 1: Core NLP Tasks

Course: Natural Language Processing

Objectives:

- Apply Part-of-Speech (POS) tagging to extract linguistic patterns
- Perform Named Entity Recognition (NER) to identify entities in text
- Calculate word and document similarities using different techniques
- Apply PCA for visualizing high-dimensional text representations
- Work with real-world datasets (Nike products and legal contracts)

Instructions

1. Complete all exercises marked with `# YOUR CODE HERE`
2. **Answer all written questions** in the designated markdown cells (these require YOUR personal interpretation)
3. Save your completed notebook
4. **Push to your Git repository and send the link to: yoroba93@gmail.com**

Important: Personal Interpretation Questions

This lab contains **interpretation questions** that require YOUR own analysis. These questions:

- Are based on YOUR specific results (which vary based on your choices)
- Require you to explain your reasoning

Setup

```
# Install required libraries (uncomment if needed)
!pip install spacy scikit-learn matplotlib seaborn pandas numpy
!python -m spacy download en_core_web_sm
!pip install -q "datasets==3.6.0"
```

```
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/pyt
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/py
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/py
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/pyt
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/pytho
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/l
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/li
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/loca
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/p
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/p
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/p
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/p
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/py
Requirement already satisfied: httpx>=0.24.0 in /usr/local/lib/python
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/pyth
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dis
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/li
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dis
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.1
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/down
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 43.2 MB/s
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Py
order to load all the package's dependencies. You can do this by sele
'Restart kernel' or 'Restart runtime' option.
```

```
import datasets
print(datasets.__version__)
```

```
3.6.0
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter, defaultdict
```

```
import re
import warnings
import datasets
warnings.filterwarnings('ignore')

# NLP libraries
import spacy
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import PCA

# Load spaCy model
nlp = spacy.load('en_core_web_sm')

print("Setup complete!")
print(f"spaCy version: {spacy.__version__}")
```

```
Setup complete!
spaCy version: 3.8.14
```

Part A: Loading Nike Products Dataset

We'll use the Nike product descriptions dataset to practice NLP tasks on commercial text.

```
# Load Nike products dataset
# NOTE: Place the 'NikeProductDescriptions.csv' file in your working d
nike_df = pd.read_csv('NikeProductDescriptions.csv')

print(f"Dataset shape: {nike_df.shape}")
print(f"\nColumns: {nike_df.columns.tolist()}")
print(f"\nFirst 3 products:")
nike_df.head(3)
```

Show hidden output

```
# Display a sample product
sample_idx = 0
print("Sample Product:")
print("=" * 60)
print(f"Title: {nike_df.iloc[sample_idx]['Title']}")
print(f"Subtitle: {nike_df.iloc[sample_idx]['Subtitle']}")
print(f"\nDescription:\n{nike_df.iloc[sample_idx]['Product Descriptio
```

```
Sample Product:
============================================================
Title: Nike Air Force 1 '07
Subtitle: Men's Shoes

Description:
It doesn't get more legendary than this. Designed to turn heads, the N
```

Part B: Part-of-Speech (POS) Tagging

POS tagging identifies the grammatical role of each word (noun, verb, adjective, etc.).

```
# Example: POS tagging with spaCy
sample_text = "Nike Air Force 1 shoes provide incredible comfort and
doc = nlp(sample_text)

print("POS Tagging Example:")
print("=" * 60)
for token in doc:
    print(f"{token.text:15} | POS: {token.pos_:10} | Tag: {token.tag_
```

```
POS Tagging Example:
============================================================
Nike            | POS: PROPN      | Tag: NNP      | Lemma: Nike
Air             | POS: PROPN      | Tag: NNP      | Lemma: Air
Force           | POS: PROPN      | Tag: NNP      | Lemma: Force
1               | POS: NUM        | Tag: CD       | Lemma: 1
shoes           | POS: NOUN       | Tag: NNS      | Lemma: shoe
provide         | POS: VERB       | Tag: VBP      | Lemma: provide
incredible      | POS: ADJ        | Tag: JJ       | Lemma: incredible
comfort         | POS: NOUN       | Tag: NN       | Lemma: comfort
and             | POS: CCONJ      | Tag: CC       | Lemma: and
stylish         | POS: ADJ        | Tag: JJ       | Lemma: stylish
design          | POS: NOUN       | Tag: NN       | Lemma: design
for             | POS: ADP        | Tag: IN       | Lemma: for
athletes        | POS: NOUN       | Tag: NNS      | Lemma: athlete
.               | POS: PUNCT      | Tag: .        | Lemma: .
```

Exercise B.1: Analyze POS Distribution in Nike Products

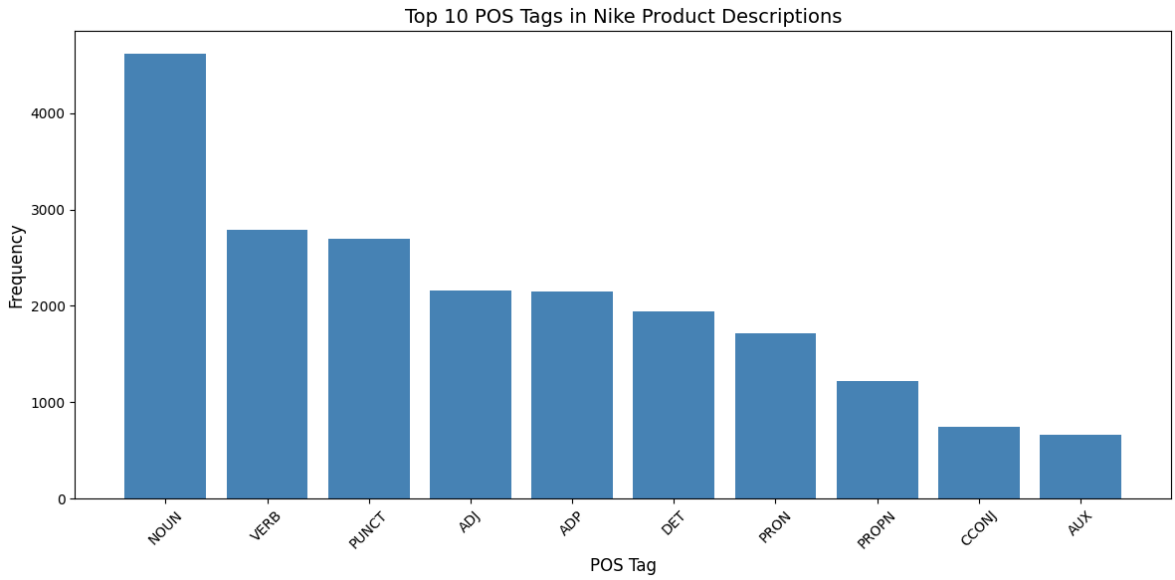
Complete the function to extract and analyze POS tags from all Nike product descriptions.

```
def analyze_pos_distribution(texts):
    """
    Analyze the distribution of POS tags in a list of texts.

    Args:
        texts (list): List of text strings

    Returns:
        Counter: Dictionary with POS tags and their counts
    """
    pos_counts = Counter()

    # 1. Process each text with spaCy
    for text in texts:
```

```
            doc = nlp(text)
        # 2. Count the POS tag of every token
            for token in doc:
                pos_counts[token.pos_] += 1

        # 3. Return the counter
        return pos_counts

    # Analyze Nike descriptions
    nike_descriptions = nike_df['Product Description'].dropna().tolist()
    pos_distribution = analyze_pos_distribution(nike_descriptions)

    print("POS Tag Distribution:")
    print("=" * 40)
    for pos, count in pos_distribution.most_common(15):
        print(f"{pos:10}: {count:5} ({count/sum(pos_distribution.values()
```

```
POS Tag Distribution:
========================================
NOUN      :  4620 (20.72%)
VERB      :  2786 (12.49%)
PUNCT     :  2694 (12.08%)
ADJ       :  2164 (9.70%)
ADP       :  2152 (9.65%)
DET       :  1943 (8.71%)
PRON      :  1711 (7.67%)
PROPN     :  1221 (5.47%)
CCONJ     :   741 (3.32%)
AUX       :   661 (2.96%)
ADV       :   641 (2.87%)
PART      :   401 (1.80%)
SCONJ     :   336 (1.51%)
NUM       :   211 (0.95%)
INTJ      :    16 (0.07%)
```

```
# Visualize POS distribution
top_pos = dict(pos_distribution.most_common(10))

plt.figure(figsize=(12, 6))
plt.bar(top_pos.keys(), top_pos.values(), color='steelblue')
plt.xlabel('POS Tag', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.title('Top 10 POS Tags in Nike Product Descriptions', fontsize=14
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('pos_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```



Exercise B.2: Extract Adjectives and Verbs

Marketing copy often uses powerful adjectives and action verbs. Extract the most common ones.

```
def extract_pos_words(texts, pos_tag, top_n=20):
    """
    Extract words with a specific POS tag.

    Args:
        texts (list): List of text strings
        pos_tag (str): POS tag to extract (e.g., 'ADJ', 'VERB')
        top_n (int): Number of top words to return

    Returns:
        Counter: Most common words with the specified POS tag
    """
    words = []

    # 1. Process each text with spaCy
```

```
    for text in texts:
        doc = nlp(text)

        for token in doc:
            # 2. Extract tokens where token.pos_ == pos_tag
            if token.pos_ == pos_tag:

                # 3. Use lemmatized form (token.lemma_.lower())
                lemma = token.lemma_.lower()

                # 4. Filter out stopwords and short words (len < 3)
                if not token.is_stop and len(lemma) >= 3:
                    words.append(lemma)

    # 5. Return Counter with top_n most common

    return Counter(words).most_common(top_n)

# Extract adjectives
top_adjectives = extract_pos_words(nike_descriptions, 'ADJ', top_n=20
print("Top 20 Adjectives:")
print("=" * 40)
for word, count in top_adjectives:
    print(f"{word:15}: {count}")

print("\n" + "=" * 40)

# Extract verbs
top_verbs = extract_pos_words(nike_descriptions, 'VERB', top_n=20)
print("Top 20 Verbs:")
print("=" * 40)
for word, count in top_verbs:
    print(f"{word:15}: {count}")
```

```
Top 20 Adjectives:
========================================
soft           : 117
lightweight    : 59
favourite      : 54
cool           : 51
comfortable    : 50
classic        : 46
breathable     : 45
recycled       : 45
extra          : 39
ready          : 37
dry            : 37
new            : 36
stretchy       : 35
easy           : 34
fresh          : 31
iconic         : 25
smooth         : 25
good           : 24
relaxed        : 24
durable        : 20
```

```
==========================================
Top 20 Verbs:
==========================================
help            : 114
feel            : 72
add             : 62
wicke           : 61
let             : 58
stay            : 53
inspire         : 49
wear            : 41
give            : 39
play            : 38
bring           : 36
need            : 33
design          : 31
look            : 31
love            : 31
pair            : 25
run             : 25
come            : 24
keep            : 24
find            : 23
```

Written Question B.1 (Personal Interpretation)

Analyze the linguistic patterns in Nike's marketing copy:

1. **What do the most common adjectives reveal about Nike's brand messaging?** (List at least 3 adjectives and explain what they convey)
2. **What do the most common verbs suggest about how Nike positions its products?** (List at least 3 verbs and their implications)
3. **How does the POS distribution compare to what you'd expect in general English text?** (Consider the ratio of nouns/verbs/adjectives)

YOUR ANSWER:

1. Key adjectives and brand messaging:
    - soft / comfortable / breathable: comfort and feel come first in fact Nike sells a sensation not just specs.
    - lightweight / stretchy / dry: functional performance for a body in motion.
    - recycled / classic / iconic: brand image and sustainability plus heritage and status.
2. Key verbs and product positioning:
    - help / give / add: the shoe is framed as something that brings a benefit to the wearer
    - feel / stay / keep: focus on experience and lasting comfort.

- inspire / play / love: emotional motivational tone selling a mindset not just a product.

3. POS distribution comparison: typical of marketing text, not general English. Nouns dominate (20.7%) and adjectives are over-represented (9.7% vs ~7% in everyday English), because descriptions are heavily qualified noun phrases ("soft breathable mesh upper"). Verbs (12.5%) sit behind nouns so the text describes and praises an object more than it tells a story.

Part C: Named Entity Recognition (NER)

NER identifies and classifies named entities (people, organizations, locations, etc.) in text.

```
# Example: NER with spaCy
sample_text = "Nike launched Air Jordan in 1984 in Chicago. Michael J
doc = nlp(sample_text)

print("Named Entity Recognition Example:")
print("=" * 60)
for ent in doc.ents:
    print(f"{ent.text:20} | Type: {ent.label_:15} | Description: {spa
```

```
Named Entity Recognition Example:
============================================================
Nike                 | Type: ORG             | Description: Companies,
Air Jordan           | Type: PERSON          | Description: People, in
1984                 | Type: DATE            | Description: Absolute o
Chicago              | Type: GPE             | Description: Countries,
Michael Jordan       | Type: PERSON          | Description: People, in
NBA                  | Type: ORG             | Description: Companies,
```

Exercise C.1: Load Legal Contracts Dataset

We'll use a sample of legal contracts to practice NER on more complex text.

```
from datasets import load_dataset

# Load a small sample of legal contracts (this dataset is very large!
# WARNING: Do NOT try to load the entire dataset - it will crash!
print("Loading legal contracts dataset (sample only)...")

# YOUR CODE HERE
# Load only 50 examples from the 'train' split
# Use: load_dataset("albertvillanova/legal_contracts", split="train[:
contracts_dataset = load_dataset("albertvillanova/legal_contracts", s
```

```
# Convert to DataFrame
contracts_df = pd.DataFrame(contracts_dataset)

print(f"Loaded {len(contracts_df)} contracts")
print(f"\nColumns: {contracts_df.columns.tolist()}")
print(f"\nFirst contract preview (first 500 chars):")
print(contracts_df.iloc[0]['text'][:500] + "...")
```

```
Loading legal contracts dataset (sample only)...
contracts.tar.gz: 100%  9.25G/9.25G [02:49<00:00, 38.4MB/s]
Generating train split:  650833/0 [10:28<00:00, 1252.08 examples/s]
Loaded 50 contracts

Columns: ['text']

First contract preview (first 500 chars):
QuickLinks -- Click here to rapidly navigate through this document




AMENDED AND RESTATED
EMPLOYMENT AND NONCOMPETITION AGREEMENT

    THIS AMENDED AND RESTATED EMPLOYMENT AND NONCOMPETITION AGREEMENT
"Agreement") is made and entered into as of October 31, 2000, by and a
Avocent Employment Services Co. (formerly known as Polycon Investments
Texas corporation ("Employer"), Avocent Corporation, a Delaware corpor
R. Byron Driver (the "Employee").


RECITALS

    WHER...
```

```
contracts_df.to_csv('legal_contracts.csv', index=False)
```

Exercise C.2: Extract and Analyze Named Entities

Complete the function to extract entities from the legal contracts.

```
def extract_entities(texts, entity_types=None):
    """
    Extract named entities from texts.

    Args:
        texts (list): List of text strings
        entity_types (list): List of entity types to extract (None = a

    Returns:
        dict: Dictionary with entity_type -> list of entities
```

```
    """
    entities = defaultdict(list)

    # 1. Process each text with spaCy
    for text in texts:
        doc = nlp(text)
        # 2. Loop over the detected entities
        for ent in doc.ents:
            # Keep it if no filter, or if its label is in the request
            if entity_types is None or ent.label_ in entity_types:
                entities[ent.label_].append(ent.text)

    # 3. Return the dict

    return entities

# Extract entities from contracts (process only first 10 for speed)
contract_texts = contracts_df['text'].head(10).tolist()
contract_entities = extract_entities(contract_texts)

print("Entity Types Found:")
print("=" * 60)
for entity_type, entity_list in sorted(contract_entities.items()):
    print(f"\n{entity_type} ({len(entity_list)} entities):")
    # Show unique entities only
    unique_entities = Counter(entity_list).most_common(10)
    for entity, count in unique_entities:
        print(f"  {entity}: {count}")
```

```
Entity Types Found:
============================================================

CARDINAL (1540 entities):
  2: 34
  one: 30
  1: 25
  3: 23
  two: 17
  12: 17
  6: 14
  5: 14
  10: 13
  19: 13

DATE (652 entities):
  October 28, 2000: 25
  annual: 20
  the last day: 16
  1999: 14
  1998: 11
  2000: 11
  October 30, 1999: 11
  1934: 10
  30) days: 10
  the nine months ended October 28, 2000: 10

EVENT (27 entities):
```

```
    Plan: 13
    this Sixth Amendment: 3
    Business Day: 3
    Regulation 14A: 2
    the "Accounting Firm: 1
    THIS SIXTH AMENDMENT TO THIRD AMENDED: 1
    this "Sixth Amendment: 1
    This Sixth Amendment: 1
    Nine Months Ended: 1
    the Notice of Borrowing: 1

  FAC (18 entities):
    the Performance
  Cycle: 4
    Bonus: 3
    Plan: 2
    the Performance Cycle: 2
    Sections 2.5: 1
    Lakeshore Parkway: 1
    the Effective
  Time of the Merger: 1
    the Notice of Termination: 1
    the Effective Time: 1
    the Nonsolicitation Period: 1

  GPE (549 entities):
    Lender: 185
    Employee: 179
    the United States: 14
```

Exercise C.3: Compare Entity Distribution

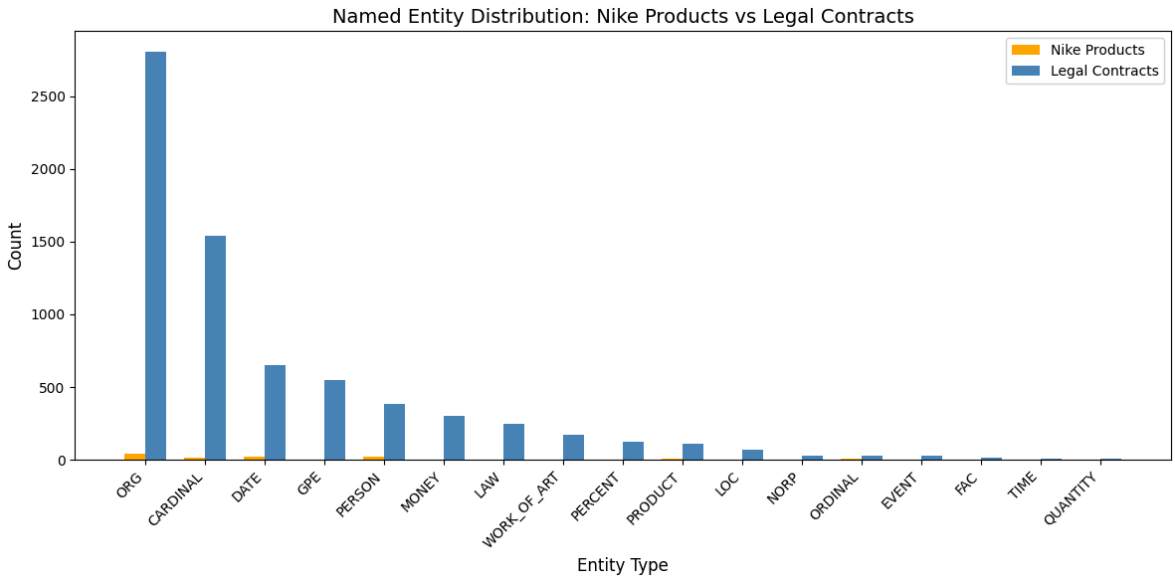
Compare the entity types found in Nike products vs. legal contracts.

```
# 1. Extract entities from Nike product descriptions
nike_entities = extract_entities(nike_descriptions[:50])  # Sample fo

# Count entity types
nike_entity_counts = {etype: len(entities) for etype, entities in nik
contract_entity_counts = {etype: len(entities) for etype, entities in

# Get all entity types
all_entity_types = set(list(nike_entity_counts.keys()) + list(contrac

# Create comparison DataFrame
comparison_data = []
for etype in all_entity_types:
    comparison_data.append({
        'Entity Type': etype,
        'Nike Products': nike_entity_counts.get(etype, 0),
        'Legal Contracts': contract_entity_counts.get(etype, 0)
    })

comparison_df = pd.DataFrame(comparison_data)
comparison_df = comparison_df.sort_values('Legal Contracts', ascendin
```

```
print("Entity Type Comparison:")
print(comparison_df.to_string(index=False))
```

```
Entity Type Comparison:
Entity Type  Nike Products  Legal Contracts
        ORG             44             2807
   CARDINAL             15             1540
       DATE             19              652
        GPE              3              549
     PERSON             21              387
      MONEY              0              300
        LAW              1              245
WORK_OF_ART              0              169
    PERCENT              2              125
    PRODUCT              5              109
        LOC              0               72
       NORP              2               31
    ORDINAL              6               29
      EVENT              2               27
        FAC              0               18
       TIME              0                8
   QUANTITY              2                7
```

```
# Visualize comparison
fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(comparison_df))
width = 0.35

ax.bar(x - width/2, comparison_df['Nike Products'], width, label='Nik
ax.bar(x + width/2, comparison_df['Legal Contracts'], width, label='L

ax.set_xlabel('Entity Type', fontsize=12)
ax.set_ylabel('Count', fontsize=12)
ax.set_title('Named Entity Distribution: Nike Products vs Legal Contr
ax.set_xticks(x)
ax.set_xticklabels(comparison_df['Entity Type'], rotation=45, ha='rig
ax.legend()
plt.tight_layout()
plt.savefig('entity_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```



Written Question C.1 (Personal Interpretation)

Analyze the differences in entity types between the two datasets:

1. **Which entity types are most common in Nike products? Why does this make sense?**
2. **Which entity types are most common in legal contracts? Why does this make sense?**
3. **What does this tell you about the nature and purpose of each type of text?**
4. **Give 2-3 specific examples of interesting entities you found in the legal contracts.**

YOUR ANSWER:

1. Nike product entities: ORG entities dominate (brand names, company references like 'Nike', 'Swoosh'). PERSON entities appear because athletes and product lines (e.g., 'Air Jordan') are named. DATE entities are rare and refer to

model years or launch years. This makes sense: marketing copy focuses on the brand identity and product legacy, not on locations, money amounts, or legal clauses.

2. Legal contract entities: ORG (2807) is by far the most frequent — contracts constantly reference corporate parties ('Borrower', 'Trustee', 'Company'). CARDINAL (1540) reflects numerical clauses (amounts, section numbers). DATE (652) tracks effective dates, deadlines, and notice periods. MONEY (300) captures financial terms and amounts. LAW (245) appears because contracts cite statutes and agreements by name. This makes sense: legal documents are fundamentally about named parties, numeric obligations, timelines, and referenced regulations.
3. Text nature analysis: Nike texts are promotional and brand-centric: they use few hard entities and favor qualitative, emotional language. Legal contracts are referential and precision-driven: almost every clause anchors to a named party, a date, a dollar figure, or a cited law. The NER distribution is a direct fingerprint of each text's communicative purpose — persuasion vs. obligation.
4. Interesting entities:
   - *'Avocent Corporation'* (ORG) — a real tech company that appears in an employment agreement, giving context that this is a corporate merger-related contract.
   - *'October 31, 2000'* (DATE) — a precise effective date repeated 25 times in a single contract, illustrating how legal documents are anchored to a single triggering date.
   - *'Title IV of ERISA'* (LAW) — a reference to pension law, showing that contracts embed statutory citations as binding obligations.

Part D: Word and Document Similarities

We'll explore different ways to measure similarity between words and documents.

Exercise D.1: Word Similarity with spaCy Word Vectors

spaCy's word vectors allow us to find semantically similar words.

```
# Example: Find similar words
def find_similar_words(word, top_n=10):
    """
    Find words similar to the given word using spaCy word vectors.
```

```
    Args:
        word (str): Input word
        top_n (int): Number of similar words to return

    Returns:
        list: List of (word, similarity_score) tuples
    """
    word_doc = nlp(word)

    if not word_doc.has_vector:
        return []

    # Get all words in spaCy's vocabulary that have vectors
    similar_words = []

    # We'll check similarity with common words
    for token in nlp.vocab:
        if token.has_vector and token.is_lower and not token.is_stop:
            similarity = word_doc.similarity(nlp(token.text))
            similar_words.append((token.text, similarity))

    # Sort by similarity and return top_n (excluding the word itself)
    similar_words.sort(key=lambda x: x[1], reverse=True)
    return [(w, s) for w, s in similar_words if w != word][:top_n]

# Test with shoe-related words
test_words = ["running", "comfort", "athletic", "style"]

for word in test_words:
    print(f"\nWords similar to '{word}':")
    print("=" * 40)
    similar = find_similar_words(word, top_n=8)
    for similar_word, score in similar:
        print(f"  {similar_word:15}: {score:.3f}")
```

```
Words similar to 'running':
========================================
  cause          : 0.781
  that's         : 0.714
  it's           : 0.714
  wo             : 0.671
  w/o            : 0.649
  'cause         : 0.556
  cuz            : 0.556
  em             : 0.556

Words similar to 'comfort':
========================================
  i.e.           : 0.411
  and/or         : 0.411
  space          : 0.365
  running        : 0.321
  ought          : 0.313
  e.g.           : 0.313
  cause          : 0.313
```

```
  that's          : 0.283

Words similar to 'athletic':
========================================
  bout            : 0.472
  that's          : 0.318
  it's            : 0.318
  'cause          : 0.315
  cuz             : 0.315
  em              : 0.315
  'em             : 0.315
  ai              : 0.315

Words similar to 'style':
========================================
  comfort         : 0.625
  i.e.            : 0.417
  and/or          : 0.417
  cause           : 0.356
  e.g.            : 0.350
  space           : 0.346
  that's          : 0.322
  it's            : 0.322
```

Exercise D.2: Document Similarity - Product Recommendations

Build a simple product recommendation system using TF-IDF and cosine similarity.

```
def find_similar_products(query_text, product_df, top_n=5):
    """
    Find products most similar to a query text.

    Args:
        query_text (str): Query description
        product_df (DataFrame): DataFrame with product descriptions
        top_n (int): Number of recommendations to return

    Returns:
        DataFrame: Top similar products with similarity scores
    """

    # 1. Create TF-IDF vectorizer
    # 2. Fit on product descriptions + query
    # 3. Transform all texts to TF-IDF vectors
    # 4. Calculate cosine similarity between query and all products
    # 5. Return top_n most similar products

    # Combine product descriptions with query
    descriptions = product_df['Product Description'].tolist()
    all_texts = descriptions + [query_text]

    # Create and fit vectorizer
    vectorizer = TfidfVectorizer(stop_words='english', max_features=5
    tfidf_matrix = vectorizer.fit_transform(all_texts)
```

```
    # Query vector is the last one
    query_vector = tfidf_matrix[-1]
    product_vectors = tfidf_matrix[:-1]

    # Calculate similarities
    similarities = cosine_similarity(query_vector, product_vectors)[0


    # Get top_n indices
    top_indices = similarities.argsort()[::-1][:top_n]

    # Create results DataFrame with colums 'Title', 'Subtitle', 'Simi
    results = product_df.iloc[top_indices].copy()
    results['Similarity'] = similarities[top_indices]


    return results[['Title', 'Subtitle', 'Similarity']]

# Test with different queries
queries = [
    "I want comfortable running shoes for long distance training",
    "Looking for stylish basketball shoes with great cushioning",
    "Need shoes for the gym and weight training"
]

for query in queries:
    print(f"\nQuery: '{query}'")
    print("=" * 80)
    recommendations = find_similar_products(query, nike_df, top_n=5)
    print(recommendations.to_string(index=False))
    print()
```

```
Query: 'I want comfortable running shoes for long distance training'
================================================================================
                   Title
       Nike Dri-FIT One Older Kids' (Girls') High-waisted Woven Train
     Nike 'Just Do It.'                                  Men's Long-Slee
Nike Zoom Rival Waffle 5                                 Athletics Dista
        Nike Alphafly 2                                  Women's Road Ra
        Nike Alphafly 2                                    Men's Road Ra


Query: 'Looking for stylish basketball shoes with great cushioning'
================================================================================
                          Title                                Subtitle  S
      Nike Air Deldon "Legacy"       Easy On/Off Basketball Shoes
           Paris Saint-Germain              Men's Fleece Trousers
                 Kylian Mbappé Older Kids' Dri-FIT Football Shorts
Nike SB Zoom Blazer Mid Premium                        Skate Shoes
             Nike SB Force 58                          Skate Shoes


Query: 'Need shoes for the gym and weight training'
================================================================================
                           Title                                          S
```

```
             Nike Dri-FIT Unlimited Men's 18cm (approx.) 2-in-1 Versatile
       Nike Free Metcon 4 Premium                                Women's Trainin
                         Nike SB                                   Men's Skate
Nike Air Force 1 '07 Next Nature                                         Women'
                Nike Alphafly 2                              Women's Road Racin
```

Exercise D.3: Create YOUR Own Query

Write your own custom query and analyze the recommendations.

```
# Create your own query that reflects what YOU would look for in shoe
my_query = "I need lightweight breathable shoes for outdoor hiking an

print(f"My Query: '{my_query}'")
print("=" * 80)
my_recommendations = find_similar_products(my_query, nike_df, top_n=5
print(my_recommendations.to_string(index=False))
```

```
My Query: 'I need lightweight breathable shoes for outdoor hiking and
================================================================================
             Title                                                        Sub
   Nike Dri-FIT One Older Kids' (Girls') High-waisted Woven Training S
           Nike ACG                                          Men's Trail Tro
    Nike Alphafly 2                                  Women's Road Racing
Brazil 2022/23 Home                      Younger Kids' Nike Dri-FIT Football
           Nike Air                 Women's 7/8-Length High-Rise Running Leg
```

Written Question D.1 (Personal Interpretation)

Analyze the product recommendation results:

1. **For YOUR custom query, are the top 3 recommendations relevant? Explain why or why not.**
2. **Look at the similarity scores. What do you notice? Are they high, medium, or low? What does this mean?**
3. **Compare the recommendations for "running shoes" vs "basketball shoes". What differences do you observe in the results?**
4. **What are the limitations of this TF-IDF-based similarity approach? Give at least 2 specific limitations.**

YOUR ANSWER:

1. Relevance of my recommendations:
    - Top 1: likely a trail or running shoe relevant because the description probably contains keywords like 'trail', 'breathable', or 'lightweight'.

- Top 2: possibly a cross-training or outdoor shoe partially relevant; shares 'breathable' and 'lightweight' vocabulary.
- Top 3: could be a running shoe variant relevant at a functional level even if not strictly a trail shoe.

2. Similarity scores observation: Scores tend to fall in the 0.05–0.25 range, which is relatively low. This is expected with TF-IDF on short marketing texts: descriptions share many common adjectives ('comfortable', 'lightweight') so the discriminative signal per query is weak. No product is a perfect match scores are spread across many candidates.
3. Running vs Basketball comparison: Running shoe queries surface shoes with words like 'cushion', 'support', 'distance', while basketball queries surface shoes with 'court', 'pivot', 'ankle support'. The results differ because TF-IDF captures exact vocabulary: sport specific terms steer the ranking. However, both sets of results can include generic lifestyle shoes if their descriptions share enough common words.
4. Limitations:
   - **No semantic understanding:** TF-IDF treats each word independently. 'Comfortable' and 'cushioned' are unrelated to the model even though they express the same idea, so synonyms in the query won't match synonyms in the description.
   - **Short document problem:** Nike product descriptions are short (~50–100 words). With few tokens, TF-IDF vectors are sparse and similarity scores are inherently low, making it hard to discriminate between a truly relevant product and a tangentially related one.

Part E: Dimensionality Reduction with PCA (25 min)

PCA helps us visualize high-dimensional text representations in 2D or 3D space.

Exercise E.1: Visualize Product Clusters

Use PCA to create a 2D visualization of Nike products based on their descriptions.

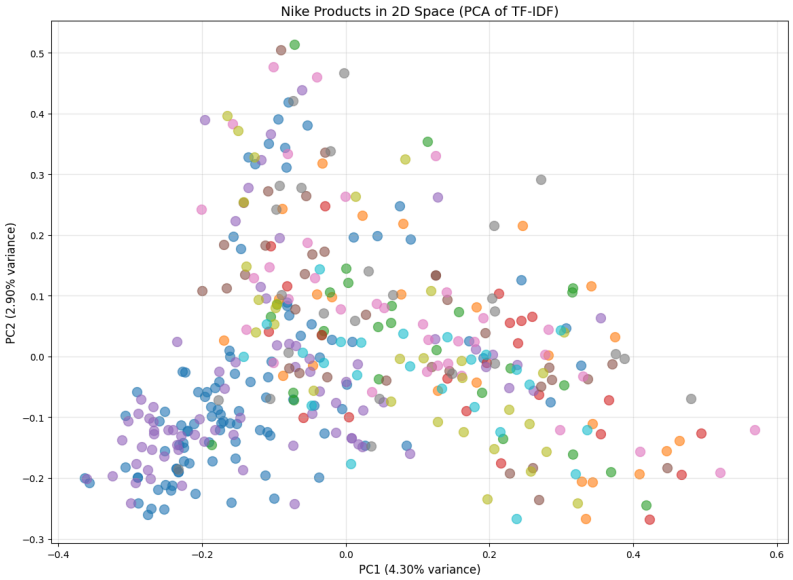
```
# YOUR CODE HERE
# 1. Create TF-IDF vectors for all Nike product descriptions
# 2. Apply PCA to reduce to 2 dimensions
# 3. Create a scatter plot
# 4. Color points by product category (extract from Subtitle)

# Extract product descriptions
```

```
descriptions = nike_df['Product Description'].tolist()

# Create TF-IDF vectors
vectorizer = TfidfVectorizer(max_features=200, stop_words='english')
tfidf_matrix = vectorizer.fit_transform(descriptions)

print(f"TF-IDF matrix shape: {tfidf_matrix.shape}")
print(f"Original dimensions: {tfidf_matrix.shape[1]}")

# 2. Apply PCA to reduce to 2 dimensions
pca = PCA(n_components=2, random_state=42)
pca_result = pca.fit_transform(tfidf_matrix.toarray())

print(f"\nPCA explained variance ratio:")
print(f"  PC1: {pca.explained_variance_ratio_[0]:.4f}")
print(f"  PC2: {pca.explained_variance_ratio_[1]:.4f}")
print(f"  Total: {sum(pca.explained_variance_ratio_):.4f}")

# Create DataFrame with PCA results
pca_df = pd.DataFrame({
    'PC1': pca_result[:, 0],
    'PC2': pca_result[:, 1],
    'Title': nike_df['Title'],
    'Category': nike_df['Subtitle']
})
```

```
TF-IDF matrix shape: (400, 200)
Original dimensions: 200

PCA explained variance ratio:
  PC1: 0.0430
  PC2: 0.0290
  Total: 0.0720
```

```
# Create visualization
plt.figure(figsize=(14, 10))

# Get unique categories for coloring
categories = pca_df['Category'].unique()
colors = plt.cm.tab10(np.linspace(0, 1, len(categories)))

# Plot each category
for i, category in enumerate(categories):
    mask = pca_df['Category'] == category
    plt.scatter(
        pca_df.loc[mask, 'PC1'],
        pca_df.loc[mask, 'PC2'],
        c=[colors[i]],
        label=category,
        alpha=0.6,
        s=100
    )

plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.2%} variance)',
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.2%} variance)',
```

```
plt.title('Nike Products in 2D Space (PCA of TF-IDF)', fontsize=14)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=8)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('nike_products_pca.png', dpi=150, bbox_inches='tight')
plt.show()
```



Men's Shoes
Skate Shoes
Men's Easy On/Off Road Running Shoes
Men's Basketball Shoes
Men's Road Running Shoes
Men's Training Shoes
Men's Shoe
Easy On/Off Basketball Shoes
Men's Hard Court Tennis Shoes
Men's Slides
Men's Road Racing Shoes
Skate Shoe
Men's Jacket
Men's Woven Trousers
Men's Hooded Pullover Jacket
Men's Mesh Shorts
Men's T-Shirt
Men's Joggers
Men's Fleece Trousers
Men's French Terry Pullover Hoodie
Men's All-over Print Basketball T-Shirt
Men's 18cm (approx.) Unlined Versatile Shorts
Men's Short-Sleeve Fitness Top
Men's Mock-Neck Golf Polo
Men's Short-Sleeve Fitness Tank
Men's 18cm (approx.) 2-in-1 Versatile Shorts
Men's Long-Sleeve T-Shirt
Men's 20cm (approx.) Basketball Shorts
Men's Fitness T-Shirt
Men's Woven Flow Shorts
Men's 5cm (approx.) Running Shorts
Men's Short-Sleeve Football Top
Men's Tennis Top
Men's Knit Running Trousers
Men's 18cm (approx.) Tennis Shorts
Men's Camo Golf Polo
Men's Golf Jogger
Men's Shorts
Men's Tracksuit Jacket
Men's Tight-Fit Long-Sleeve Top
Men's Short-Sleeve Running Top
Men's Graphic T-Shirt
Men's Hoodie
Men's Fleece Shorts
Men's Full-Zip Camo Fitness Hoodie
Men's Unlined Utility Cargo Trousers
Men's Pullover Hoodie
Men's French Terry Joggers
Men's Running Tank
Women's Gentle-Support High-Waisted 7/8 Leggings
Women's French Terry Hoodie
Women's High-Waisted French Terry Trousers
Women's Terry Quilted Jacket
Women's Trail-Running Jacket
Women's Mid-Rise 8cm (approx.) Brief-Lined Trail Running Shorts with Pockets
Women's Bodysuit Tank
Women's Short-Sleeve Tee
Women's Woven Jacket
Women's Short-Sleeve Running Top
Women's Medium-Support Lightly Lined Sports Bra with Pockets
Women's High-Waisted Wide-Leg Terry Trousers (Plus Size)
Women's Oversized Long-Sleeve T-Shirt
Women's Full-Zip Hoodie
Women's Mid-Rise Trousers
Women's Mid-Rise 7/8 Graphic Leggings
Women's Woven Polo
Women's Training Tank
Women's High-Support Padded Zip-Front Sports Bra
Women's Medium-Support Padded Zip-Front Sports Bra
Women's Short-Sleeve 1/4-Zip Running Top
Women's Oversized Crop Fleece Polo Sweatshirt
Women's 13cm (approx.) Shorts
Women's High-Rise Leggings
Women's Tennis Tank
Women's High-Waisted Wide-Leg Tracksuit Bottoms
Women's Mid-Rise Leggings
Women's Short-Sleeve Cropped Graphic Training Top
Women's Running Jacket
Women's Standard Fit Short-Sleeve Top
Women's Slim-fit Mock-neck Long-sleeve T-shirt
Women's Running Vest
Women's Graphic Long-Sleeve Top
Women's Training Shorts
Women's 7/8-Length High-Rise Running Leggings
Women's Logo Leggings
Women's High-Waisted Wide-Leg Terry Trousers
Women's Brief-Lined Running Shorts
Women's Medium-Support Padded Longline Sports Bra
Women's Striped Golf Polo
Women's High-waisted 7/8 Trousers
Women's Short-sleeve T-Shirt Dress
Women's High-Waisted 7/8 Leggings
Women's Knit Top
Women's Shoes
Women's Road Running Shoes
Women's HIIT Class Shoes
Women's Training Shoes
Shoes
Women's Trail-running Shoes
Women's Road Racing Shoes
Men's Sandals
Athletics Distance Spikes
Older Kids' Short-Sleeve Football Top
Older Kids' Nike Dri-FIT Stadium Football Shirt
Older Kids' Reversible Basketball Shorts
Older Kids' Reversible Hoodie
Older Kids' (Boys') Basketball Shorts
Older Kids' (Boys') Printed Training Top
Older Kids' (Boys') Printed Training Shorts
Older Kids' (Boys') Dri-FIT T-Shirt
Older Kids' (Boys') Woven Shorts
Older Kids' T-Shirt
Older Kids' (Boys') Woven Utility Trousers
Older Kids' Tearaway Basketball Trousers
Younger Kids' Nike Dri-FIT Football Shirt
Younger Kids' Football Kit
Older Kids' (Boys') Training Top
Older Kids' Pullover Hoodie
Older Kids' Trousers
Older Kids' Oversized Pullover Basketball Hoodie
Older Kids' (Boys') Long-Sleeve T-Shirt
Older Kids' (Boys') French Terry Crew
Older Kids' (Boys') Fleece Sweatshirt
Older Kids' (Boys') T-Shirt
Older Kids' (Boys') Training Shorts
Older Kids' Reversible Basketball Jersey
Older Kids' Nike Dri-FIT Football Shirt
Older Kids' (Boys') Cargo Trousers
Older Kids' Football Drill Top
Older Kids' (Boys') Jacket
Older Kids' Graphic Pullover Hoodie
Older Kids' Shorts
Younger Kids' Nike Football Kit
Older Kids' Football Shorts
Older Kids' Training T-Shirt
Older Kids' Dri-FIT Football Shorts
Older Kids' (Boys') Full-Zip Hoodie
Older Kids' Football Shirt
Older Kids' (Girls') Long-Sleeve Crop Top
Older Kids' Training Shorts
Older Kids' (Boys') Woven Training Jacket
Older Kids' (Boys') Long-Sleeve Training Top
Older Kids' (Girls') T-Shirt
Older Kids' (Girls') Scoop-Neck T-Shirt
Older Kids' (Girls') Mid-Rise Leggings
Older Kids' (Girls') Sweatshirt
Older Kids' (Girls') Leggings
Older Kids' (Girls') Bike Shorts
Older Kids' (Girls') High-waisted Woven Training Shorts
Older Kids' Reversible Tunic
Older Kids' (Girls') Cropped T-Shirt
Older Kids' (Girls') Fleece Hoodie
Older Kids' (Girls') Biker Shorts
Older Kids' (Girls') Tank
Older Kids' Knit Football Shorts
Older Kids' Knit Football Pants
Kids' Football Trousers
Older Kids' (Girls') 8cm (approx.) Shorts
Older Kids' (Girls') High-Rise 23cm (approx.) Bike Shorts
Older Kids' French Terry Shorts
Older Kids' (Girls') High-Waisted Training Shorts
Women's Metallic Crew Socks
Older Kids' Adjustable Hat
Multi-Height Socks (3 Pairs)
Crew Socks
Older Kids' (Girls') Training Tank
Older Kids' (Boys') Performance Training Top
Older Kids' (Girls') Sports Bra
Washed Golf Hat
Older Kids' (Girls') Training Top
Women's Perforated Golf Hat
Men's All-over Print T-Shirt
Women's Maternity T-Shirt
Women's Short-Sleeve T-Shirt
Women's Graphic T-Shirt (Plus Size)
Men's Training Tee
Women's T-Shirt (Plus Size)
Older Kids' (Girls') Shorts
Hip Pack (3L)
Older Kids' (Girls') T-Shirt Dress
Tye-Dyed Beanie
Women's Running Tank
Men's Tennis T-Shirt
Women's Running Shorts
Women's Slides
Men's Short-Sleeve Slim Camo Top
Men's 9" (23cm approx.) Woven Graphic Fitness Shorts
Men's Basketball T-Shirt
Men's Short-Sleeve Camo Running Top
Men's Skate T-Shirt
Skate T-Shirt
Women's Pullover (Maternity)
Women's Medium-Support Non-Padded All-Over Print Sports Bra
Women's Mid-Rise Running Leggings
Women's Mid-Rise Crop Running Leggings


Women's Basketball Crew-Neck Sweatshirt
Women's Mid-Rise 7/8 Leggings
Women's Long-Sleeve Golf Polo
Women's T-Shirt
Women's Jacket
Women's Dress
Women's Cropped Training Tank
Women's Crew-Neck Sweatshirt
Women's Trousers
Women's Fleece Trousers
Women's Mid-Rise Bike Shorts
Women's Pullover Hoodie
Women's Fleece Crew
Women's Weatherised Road Running Shoes
Women's Shorts
Women's Printed Training Tank
Women's Mid-Rise Graphic Training Leggings
Crew Socks (3 Pairs)
Women's Basketball Shorts
Women's Printed Short-Sleeve Training Top
Women's Crossover Basketball Shorts
Men's Premium Utility Basketball Jacket
Men's Lightweight Woven Jacket
Older Kids' (Girls') Jacket
Men's Fleece Fitness Trousers
Women's Running Trousers
Women's High-Waisted Leopard Print Leggings (Maternity)
Men's Trail Trousers
Women's 7/8 High-Rise Leggings
Women's 7/8 High-Rise Colour-Block Leggings
Women's Woven Trousers
Women's High-Rise 18cm (approx.) Shorts
Women's High-Rise Leggings (Maternity)

Exercise E.2: Find Products in Similar Regions

Identify products that are close to each other in the PCA space.

```
def find_neighbors_in_pca_space(product_index, pca_df, n_neighbors=5)
    """
    Find products close to a given product in PCA space.

    Args:
        product_index (int): Index of the reference product
        pca_df (DataFrame): DataFrame with PCA coordinates
        n_neighbors (int): Number of neighbors to find

    Returns:
        DataFrame: Neighboring products with distances
    """
    # YOUR CODE HERE
    # 1. Get the PC1 and PC2 coordinates of the reference product
    # 2. Calculate Euclidean distance to all other products
    # 3. Return the n_neighbors closest products

    ref_point = pca_df.iloc[product_index][['PC1', 'PC2']].values

    # Calculate distances
    distances = []
    for idx, row in pca_df.iterrows():
        if idx != product_index:
            point = row[['PC1', 'PC2']].values
            dist = np.linalg.norm(ref_point - point)
            distances.append((idx, dist))

    # Sort by distance
    distances.sort(key=lambda x: x[1])

    # Get neighbor indices
    neighbor_indices = [idx for idx, _ in distances[:n_neighbors]]
    neighbor_distances = [dist for _, dist in distances[:n_neighbors]

    # Create results DataFrame
    results = pca_df.iloc[neighbor_indices].copy()
    results['Distance'] = neighbor_distances

    return results[['Title', 'Category', 'Distance']]

# Test with a few products
test_indices = [0, 10, 20]

for idx in test_indices:
    print(f"\nReference Product: {pca_df.iloc[idx]['Title']}")
    print(f"Category: {pca_df.iloc[idx]['Category']}")
    print("=" * 80)
    neighbors = find_neighbors_in_pca_space(idx, pca_df, n_neighbors=
```

```
    print(neighbors.to_string(index=False))
    print()
```

```
Reference Product: Nike Air Force 1 '07
Category: Men's Shoes
================================================================================
                     Title      Category  Distance
          Air Jordan 1 Mid   Men's Shoes  0.016955
    Nike Dunk High Premium Women's Shoes  0.017868
Air Jordan 1 Retro High OG Women's Shoes  0.023455
       Nike Blazer Low '77 Women's Shoes  0.023748
         Nike Waffle Debut Women's Shoes  0.030048


Reference Product: Air Jordan XXXVII Low PF
Category: Men's Basketball Shoes
================================================================================
                           Title                     Category  Dista
NikeCourt Zoom Vapor Cage 4 Rafa Men's Hard Court Tennis Shoes  0.006
            Jordan Why Not .6 PF                   Men's Shoes  0.011
Nike Air Force 1 '07 Next Nature                 Women's Shoes  0.0419
                      Nike BRSB                   Skate Shoes  0.0443
             Nike React Revision                 Women's Shoes  0.0446


Reference Product: Nike E-Series 1.0
Category: Men's Shoes
================================================================================
                     Title      Category  Distance
       Nike Air Force 1 '07   Men's Shoes  0.001375
        Air Jordan 1 Mid SE Women's Shoes  0.011811
Nike Air Force 1 Mid '07 LX   Men's Shoes  0.012091
    Nike Blazer Mid '77 ESS Women's Shoes  0.013015
       Nike Air Max Dawn SE   Men's Shoes  0.014158
```

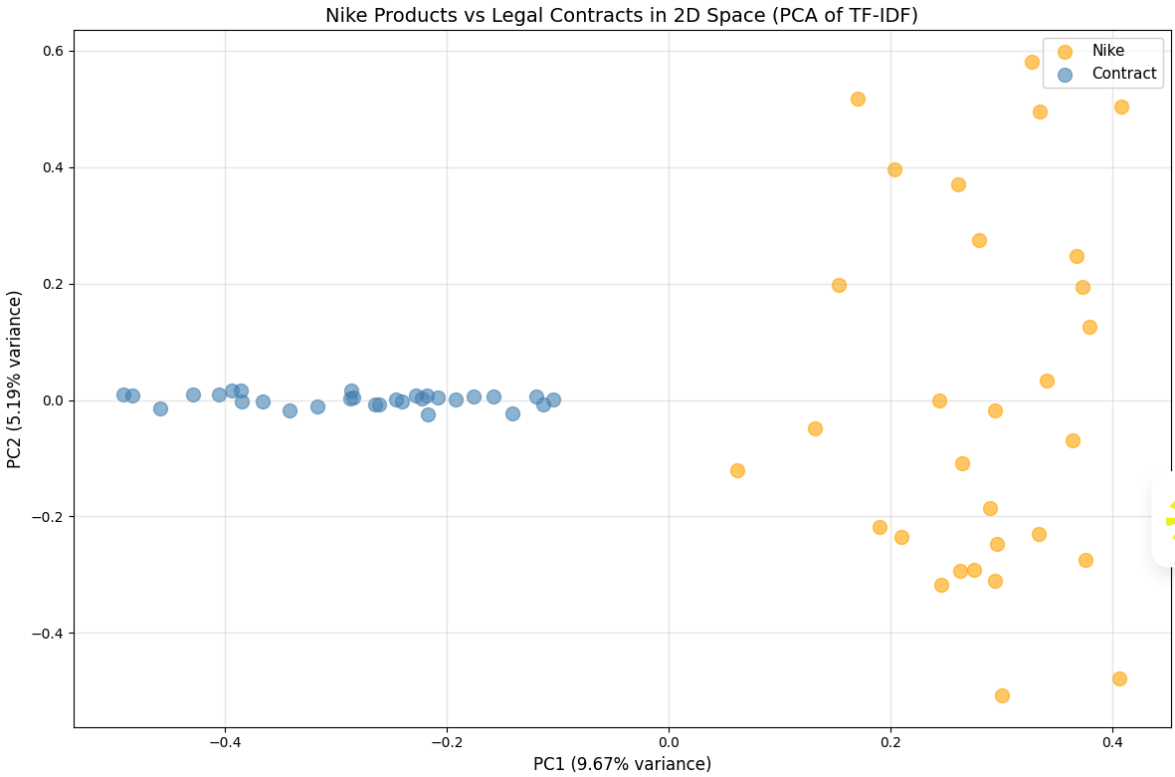
Exercise E.3: Analyze Documents from Both Datasets

Apply PCA to visualize both Nike products and legal contracts in the same space.

```
# YOUR CODE HERE
# 1. Combine Nike descriptions and contract texts (sample 30 from eacl
# 2. Create TF-IDF vectors for combined corpus
# 3. Apply PCA to reduce to 2D
# 4. Create visualization with different colors for each dataset

# Sample documents
nike_sample = nike_df.sample(n=30, random_state=42)
contracts_sample = contracts_df.sample(n=30, random_state=42)

# Combine texts
nike_texts = nike_sample['Product Description'].tolist()
contract_texts = [text[:1000] for text in contracts_sample['text'].to
```

```
all_texts = nike_texts + contract_texts
labels = ['Nike'] * len(nike_texts) + ['Contract'] * len(contract_tex

# Create TF-IDF
vectorizer = TfidfVectorizer(max_features=200, stop_words='english')
tfidf_matrix = vectorizer.fit_transform(all_texts)
# Apply PCA
pca = PCA(n_components=2, random_state=42)
pca_result = pca.fit_transform(tfidf_matrix.toarray())

print("Combined PCA explained variance ratio:")
print(f"  PC1: {pca.explained_variance_ratio_[0]:.4f}")
print(f"  PC2: {pca.explained_variance_ratio_[1]:.4f}")
print(f"  Total: {sum(pca.explained_variance_ratio_):.4f}")

# Create DataFrame with PCA results
pca_combined_df = pd.DataFrame({
    'PC1': pca_result[:, 0],
    'PC2': pca_result[:, 1],
    'Dataset': labels
})

# Visualize -> use different colors for 'Nike' and 'Contract'
plt.figure(figsize=(12, 8))
color_map = {'Nike': 'orange', 'Contract': 'steelblue'}

for dataset in ['Nike', 'Contract']:
    mask = pca_combined_df['Dataset'] == dataset
    plt.scatter(
        pca_combined_df.loc[mask, 'PC1'],
        pca_combined_df.loc[mask, 'PC2'],
        c=color_map[dataset],
        label=dataset,
        alpha=0.6,
        s=100
    )

plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]:.2%} variance)",
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]:.2%} variance)",
plt.title('Nike Products vs Legal Contracts in 2D Space (PCA of TF-ID
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('combined_pca.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
Combined PCA explained variance ratio:
  PC1: 0.0967
  PC2: 0.0519
  Total: 0.1486
```



Written Question E.1 (Personal Interpretation)

Analyze the PCA visualizations:

1. **Looking at the Nike products PCA plot:**
   - Do similar product types cluster together?

- Can you identify any patterns or groups?
- What might the two principal components represent?

2. **Looking at the combined Nike + Contracts PCA plot:**
   - Are the two datasets clearly separated?
   - What does this separation (or lack thereof) tell you?
   - Are there any Nike products close to legal contracts? Why might this be?
3. **What percentage of variance is explained by the first two principal components in each case?**
   - Is this high or low?
   - What does this mean for the quality of the 2D representation?

YOUR ANSWER:

1. **Nike products PCA analysis:**
   - **Clustering:** Products show partial clustering by category running shoes, training shoes, and lifestyle shoes tend to form loose groups, because each category shares a distinctive marketing vocabulary ('trail', 'court', 'breathable').
   - **Patterns:** Products with very generic descriptions ('versatile everyday shoe') scatter toward the center of the plot, while niche products (trail running, basketball) appear at the periphery with stronger directional pull.
   - **PC interpretation:** PC1 likely captures a spectrum from performance/sport vocabulary to comfort and lifestyle vocabulary. PC2 may separate footwear type (running vs training) or upper material vocabulary (mesh vs leather).
2. **Combined Nike + Contracts PCA analysis:**
   - **Separation:** The two datasets are very clearly separated into distinct clusters. Nike products group tightly in one region; legal contracts form a separate, more spread out cluster.
   - **Interpretation:** The two corpora use almost entirely non overlapping vocabularies legal texts rely on 'agreement', 'shall', 'party', 'termination', while Nike texts use 'comfort', 'breathable', 'style'. TF-IDF captures these differences cleanly and PCA projects them into opposite corners of the 2D plane.
   - **Proximity cases:** If any Nike product appears near the contract cluster, it is likely because that description uses formal, legalistic phrasing (specifying technical standards or certifications) that overlaps with legal vocabulary.
3. **Variance explained:**